loader.py

`src/moe_pruning/model/loader.py · 198 lines · module moe_pruning.model.loader`

What it does

Loads the original model, its tokenizer and plugin, and gives lazy access to the original checkpoint's tensors.

How it works

- `load_original_model()` selects the plugin from the model's config, loads with the configured dtype, cache directory, experts implementation and placement (one device or a multi-GPU device map) and returns the device inputs must go to.
- `load_tokenizer()` loads the tokenizer and sets a pad token if the model has none (padding is always masked).
- `CheckpointReader` reads named tensors from safetensors shards or `pytorch_model*.bin` without loading the whole model; `StateDictReader` does the same for an in-memory state dict.
- `open_original_checkpoint()` finds the checkpoint files locally or in the Hugging Face cache.
- `load_pruned_for_config()` loads a pruned checkpoint with the configured placement and base model.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class LoadedModel`	The loaded model, tokenizer, input device, dtype, source and plugin.
`hub_kwargs(cfg)`	`revision` / `cache_dir` for every Hugging Face download (HF_CACHE_DIR).
`model_load_kwargs(cfg)`	Extra `from_pretrained` kwargs common to all model loads.
`load_tokenizer(cfg)`	Load the tokenizer (sets a pad token if the model has none).
`get_model_plugin(cfg)`	Model plugin of the configured model (reads only its config.json).
`load_original_model(cfg, device)`	Load the unmodified model in evaluation mode on its configured device(s).
`class CheckpointReader`	Lazy, memory-mapped access to the tensors of the ORIGINAL checkpoint on disk.
`.keys()`	Names of all tensors in the checkpoint.
`.has(name)`	True if a tensor with this name exists.
`.get(name)`	Read one tensor.
`open_original_checkpoint(cfg)`	CheckpointReader for the configured original model (local dir or HF cache).
`class StateDictReader`	CheckpointReader over an in-memory state dict (tests, already-loaded weights).
`.keys()`	Names of all tensors.
`.has(name)`	True if a tensor with this name exists.
`.get(name)`	Copy of one tensor.
`load_pruned_for_config(cfg, model_path)`	`load_pruned_model` with the configured placement, cache dir and base model.

Settings it reads

`EXPERTS_IMPLEMENTATION`, `HF_CACHE_DIR`, `HF_MODEL_ID`, `MODEL_DTYPE`

Connections

Uses: `moe_pruning.config`, `moe_pruning.model.placement`, `moe_pruning.models`, `moe_pruning.serialization.checkpoint`, `moe_pruning.utils.device`

Used by: `scripts/collect_stats.py`, `scripts/convert_model.py`, `scripts/evaluate.py`, `scripts/export_model.py`, `scripts/inspect_model.py`, `scripts/replace_experts.py`, `scripts/train_biases.py`